LESSON 10 · MISSION GUIDE

Pivot Season

groupby is the PivotTable, merge is XLOOKUP. Together they are most of the pay gap report.

From **Gerald Brightwell, Managing Director**
To **Jen**
Subject **Board pack (sent Friday 16:55)**

Jen, the board meets in three weeks and wants the pay gap "by department, by site, by grade, with the bonus thing". Last year Siobhan built eleven PivotTables by hand and one pointed at the wrong tab. We found out in the meeting.

Mo says pandas does a PivotTable in one line (`groupby`) and XLOOKUP in one line (`merge`), so the eleven tables would be eleven lines. If that's true, please build them. If not, tell Mo before the meeting, not during. No pressure. (Some pressure.) — G.

What you'll be able to do after today

- [ ] Count things by category with `value_counts()`
- [ ] Build a **PivotTable** with `groupby`: one or two row fields, any summary (mean, median, count, sum)
- [ ] Build the same with `pivot_table`, rows down, columns across, grand totals
- [ ] **XLOOKUP** a second table in with `merge`, and spot the rows it didn't find (NaN)
- [ ] Find and fill **missing values**: `.isna()`, `.fillna()`
- [ ] Cut people into **quartiles** with `qcut` and cross-tabulate them

Why this matters at work

- **The pay gap by department, site and grade is a groupby.** Exercise 3 produces the table the board wants, worst first.
- **Quartiles and the bonus gap** (Exercises 6 and 7) are the remaining numbers in the statutory report. After today you have all six.

Four big ideas

1. groupby = PivotTable	`df.groupby("department")["annual_salary"].mean()`: row field, values, summary. Two row fields: `groupby(["a", "b"])`, a list.
2. unstack / pivot_table	Turn the second row field into columns. `pivot_table(index=, columns=, values=, aggfunc=)` is the PivotTable dialog with names.
3. merge = XLOOKUP for whole tables	`df.merge(other, on="key", how="left")`. Left keeps everyone; unmatched rows get NaN. Inner keeps only matches and silently drops the rest.
4. NaN is "no value"	A failed lookup or an empty cell. `.isna().sum()` counts them, `.fillna(x)` replaces them.

Three things that will bite you

- `groupby("a", "b")` is not two keys. `groupby(["a", "b"])` is.
- `how="inner"` loses rows without a word. Print `len()` before and after every merge.
- Rounding before grouping gives slightly different means than rounding after. Round at the end.

How today works

15 MIN
Lecture
Lars rebuilds Siobhan's eleven PivotTables, live

60 MIN
Exercise sheet
Seven tickets: the board pack, table by table

15 MIN
Dave's Leftovers #10
"pivot_magic.py", recovered from a locked tab

The inbox

#	FROM	TICKET	YOU'LL USE
1	Siobhan	Headcount	`value_counts()`, `normalize=True`
2	Gerald	The PivotTable	`groupby().mean()` `.agg([...])` `.idxmax()`
3	Priya	Two row fields	`groupby([a, b])`, `.unstack()`
4	Gerald	pivot_table, the real one	`pivot_table(..., margins=True)`
5	Mo	XLOOKUP	`merge()`, `.isna()` `.fillna()`
6	Gerald	Pay quartiles	`pd.qcut()`, `pd.crosstab()`
7 ⭐	Siobhan	The bonus gap	all of the above, `.loc[row, col]`

You're done when...

- [ ] `my_work/` has `ex1` to `ex6` (and `ex7` if Siobhan's 2022 file can stay lost), each printing the expected output
- [ ] Dave's pivot_magic runs with all 200 people and the right percentages, one commit per bug
- [ ] Everything is pushed to `lesson-10` and you've opened a pull request



Next lesson: Show Gerald. Charts with matplotlib and a properly formatted Excel workbook written by pandas. The numbers are done; now they have to look like something the board will read.